# Wprowadzenie do programowania w języku Python

### Sesja 1: Fundamenty

dr inż. Leszek Tarkowski

## Plan sesji

1. Wprowadzenie do programowania
2. Instalacja i konfiguracja środowiska
3. Zmienne i typy proste
4. Kolekcje: listy, krotki, słowniki, zbiory
5. Instrukcje warunkowe
6. Pętle

Kod piszemy razem w VS Code. Slajdy są konspektem, nie zastępują pisania.

Blok 1.6 jest buforem. Jeśli nie zmieści się dziś, otwiera sesję 2 (7.11).

## Dane, z którymi będziemy pracować

Przez cały kurs wracamy do jednego zbioru: **pasażerowie Titanica** (titanic3).

- 1309 pasażerów, 14 kolumn
- klasa, przeżycie, imię i nazwisko, płeć, wiek, bilet, opłata, kabina, port
- realne braki danych: wiek znany dla ok. 80% pasażerów

Dziś: jeden pasażer jako słownik. Sesja 2: klasa `Passenger`. Sesja 3: wykresy.

Źródło: F. E. Harrell Jr., T. Cason, Vanderbilt Biostatistics, hbiostat.org/data

## 1.1 Wprowadzenie do programowania

### Algorytm

**Algorytm** to skończony, jednoznaczny ciąg kroków prowadzący od danych do wyniku.

Przykład: czy pasażer jest dzieckiem?

1. Odczytaj wiek pasażera.
2. Jeśli wieku brak, odpowiedz „nie wiadomo”.
3. Jeśli wiek jest mniejszy niż 18, odpowiedz „tak”.
4. W przeciwnym razie odpowiedz „nie”.

Zwrócić uwagę na krok 2: przypadek brzegowy (brak danych) trzeba przewidzieć. To motyw, który wróci przy None i wyjątkach.

### Program

**Program** to algorytm zapisany w języku, który komputer potrafi wykonać.

```
age = 29

if age is None:
    print("nie wiadomo")
elif age < 18:
    print("tak")
else:
    print("nie")
```

Ten sam algorytm, cztery kroki, cztery fragmenty kodu.

### Poziomy języków

| Poziom | Przykład | Kto czyta |
|---|---|---|
| Kod maszynowy | `10110000 01100001` | procesor |
| Asembler | `mov al, 61h` | specjaliści |
| Języki kompilowane | C, C++, Rust | programiści |
| Języki interpretowane | Python, JavaScript | programiści i analitycy |

Im wyższy poziom, tym bliżej języka problemu, a dalej od sprzętu.

## Kompilacja a interpretacja

**Kompilacja:** cały program jest tłumaczony z góry na kod maszynowy; uruchamiamy gotowy plik wykonywalny.

**Interpretacja:** program jest wykonywany instrukcja po instrukcji przez inny program, interpreter.

Python (CPython) łączy oba podejścia:

```
plik .py  →  bytecode (.pyc)  →  maszyna wirtualna Pythona
```

Kompilacja do bytecode'u dzieje się automatycznie i niezauważalnie.

## Zajrzyjmy pod maskę

```
import dis

dis.dis("cena * 1.23")
```

```
  LOAD_NAME     0 (cena)
  LOAD_CONST    0 (1.23)
  BINARY_OP     5 (*)
  RETURN_VALUE
```

Każda linia to jedna instrukcja maszyny wirtualnej.

Dokładna postać wydruku zależy od wersji Pythona.

Tylko intuicja: "Python też jest tłumaczony", nie wchodzimy w szczegóły. Pokazać na żywo w REPL.

## Dlaczego Python?

- **Czytelność**: kod przypomina pseudokod.
- **Uniwersalność**: analiza danych, automatyzacja, aplikacje WWW, nauka, AI.
- **Ekosystem**: setki tysięcy pakietów w PyPI.
- **Biblioteka standardowa**: „baterie w zestawie", czyli dużo gotowych modułów od razu po instalacji.

```
import this
```

Zen of Python: przeczytać 2–3 linijki, np. “Readability counts”, “Errors should never pass silently”.

### Python w zarządzaniu

- **Analiza danych**: od pliku CSV z systemu do wniosków i wykresów.
- **Automatyzacja**: setki plików Excela, PDF-y, e-maile, raporty cykliczne.
- **Raportowanie**: wykresy i zestawienia generowane jednym poleceniem.
- **Współpraca z AI**: asystenci piszą kod, ale ktoś musi go rozumieć i sprawdzić.

Celem kursu nie jest zostanie programistą, tylko umiejętność czytania, pisania i weryfikacji prostych programów.

### Ekosystem i wersje

- **PyPI** (pypi.org): repozytorium pakietów, instalowanych przez `pip`.
- **Python 3.x**: nowa wersja co roku, w październiku.
- Każda wersja jest wspierana ok. 5 lat.
- Python 2 nie jest rozwijany od 2020 r.; starsze poradniki w sieci bywają nieaktualne.

Aktualna wersja: python.org/downloads

### Po co uczyć się programowania, skoro jest AI?

AI przesuwa wąskie gardło **z pisania kodu na jego ocenę**.

- Model pisze kod płynnie i z tą samą pewnością siebie, gdy jest poprawny i gdy jest błędny.
- Kto nie umie czytać kodu, nie odróżni jednego od drugiego.
- Kto nie odróżnia, nie może brać odpowiedzialności za wynik.

Programowanie przestaje być umiejętnością produkcji, a staje się umiejętnością **audytu**. Ta druga zyskuje na wartości.

### Prompt to życzenie, program to definicja

„Policz średni wiek pasażerów Titanica.”

```
ages = [p["age"] for p in passengers if p["age"] is not None]
average = sum(ages) / len(ages)
```

Kod musi rozstrzygnąć to, czego zdanie nie mówi:

- Brakujący wiek pominąć, czy liczyć jako 0?
- Niemowlę w wieku 0,92 roku to poprawna wartość czy błąd?
- Co, jeśli lista jest pusta?

Odpowiedź na pierwsze pytanie zmienia wynik o kilka lat: w titanic3 brakuje wieku dla ok. 20% pasażerów. Programowanie uczy kategorii (typy, warunki brzegowe), w których da się precyzyjnie powiedzieć AI, czego się chce.

### Wynik, który da się sprawdzić

| | Rozmowa z chatbotem | Skrypt |
|---|---|---|
| Uruchomić ponownie | nie | tak |
| Dać komuś do sprawdzenia | trudno | tak |
| Śledzić zmiany (wersjonować) | nie | tak |
| Wykazać, skąd wziął się wynik | nie | tak |

„Dostałem wynik" a „mogę wykazać, skąd wziął się wynik": w biznesie, nauce i administracji to zasadnicza różnica.

### Python to język samego AI

- Dostęp do modeli przez API, agenci, przetwarzanie danych, notatniki: wszystko to w praktyce Python.
- Kto zna Pythona, używa AI jako **składnika** własnych rozwiązań.
- Kto nie zna, używa AI jako **czarnej skrzynki** z okienkiem czatu.

### Lekcja z kalkulatora

Kalkulator nie zniósł potrzeby rozumienia arytmetyki. Zniósł potrzebę liczenia na kartce.

- Ci, którzy rozumieli, zaczęli liczyć więcej i trudniejsze rzeczy.
- Ci, którzy nie rozumieli, zaczęli ufać wynikom, których nie umieli sprawdzić.

AI obniżyło koszt nauki Pythona (prywatny tutor przez całą dobę), ale nie obniżyło wartości jego znajomości. Rzadko inwestycja ma tak korzystny stosunek nakładu do zwrotu.

## 1.2 Instalacja i konfiguracja środowiska

### Czego potrzebujemy

| Składnik | Do czego |
|---|---|
| Interpreter Pythona | wykonuje programy |
| VS Code + rozszerzenie Python | pisanie, uruchamianie, debugowanie |
| Jupyter / Google Colab | eksperymenty, notatniki z wynikami |

Na laboratoriach można pracować lokalnie albo w Colab (bez instalacji).

### Instalacja interpretera

**Windows:** instalator z python.org

- zaznaczyć **„Add python.exe to PATH"**
- w terminalu działa polecenie `py` lub `python`

**macOS:** instalator z python.org lub Homebrew (`brew install python`)

- w terminalu: `python3`

**Alternatywa:** `uv`, czyli szybki menedżer Pythona i pakietów

```
uv python install
```

### Sprawdzenie instalacji

```
python --version      # Windows
python3 --version     # macOS, Linux
```

Uruchomienie interaktywnej konsoli (REPL):

```
$ python3
>>> 2 + 2
4
>>> exit()
```

**REPL** = *Read, Eval, Print, Loop*: wczytaj, wykonaj, wypisz, powtórz.

## VS Code

1. Zainstalować VS Code (code.visualstudio.com).
2. Zainstalować rozszerzenie **Python** (Microsoft).
3. Wybrać interpreter: `Ctrl+Shift+P` → *Python: Select Interpreter*.
4. Utworzyć plik `.py` i uruchomić przyciskiem  lub w terminalu.

Na macOS zamiast `Ctrl` używamy `Cmd`.

## Notatniki: Jupyter i Colab

- Kod podzielony na **komórki**, wynik pod każdą komórką.
- Dobre do eksperymentów i analizy danych krok po kroku.
- **Google Colab** (colab.research.google.com): notatnik w przeglądarce, bez instalacji.

| Skrypt `.py` | Notatnik `.ipynb` |
|---|---|
| program uruchamiany w całości | kod wykonywany fragmentami |
| automatyzacja, narzędzia | eksploracja, prezentacja wyników |

## Pakiety i środowiska wirtualne

Pakiety spoza biblioteki standardowej instalujemy przez `pip`:

```
python -m pip install matplotlib
```

**Środowisko wirtualne** to osobny „katalog z Pythonem” dla jednego projektu:

```
python -m venv .venv
source .venv/bin/activate      # macOS, Linux
.venv\Scripts\activate         # Windows
```

Po co? Różne projekty mogą potrzebować różnych wersji tych samych pakietów.

## Pierwszy program

```
# hello.py: pierwszy program
print("Witaj, Python!")
```

```
python hello.py
```

- `#` rozpoczyna **komentarz**, ignorowany przez interpreter.
- `print()` to **funkcja**: nazwa, nawiasy, argumenty.

### Styl kodu: PEP 8

PEP 8 to oficjalny przewodnik stylu Pythona.

- wcięcia: **4 spacje**
- nazwy zmiennych i funkcji: `snake_case`, np. `ticket_price`
- spacje wokół operatorów: `x = a + b`, nie `x=a+b`
- linie do ok. 79–99 znaków

Kod czyta się znacznie częściej, niż się go pisze.

### Pokaz w VS Code

- Utworzyć folder `python-course` i otworzyć go w VS Code.
- Wybrać interpreter.
- Utworzyć `hello.py`, uruchomić przyciskiem  i z terminala.
- Uruchomić REPL w terminalu VS Code: `2 ** 100`, `import this`.

## 1.3 Zmienne i typy proste

### Zmienna to nazwa

```
age = 29
```

- Po prawej stronie powstaje **obiekt**: liczba 29.
- Po lewej stronie **nazwa** `age` zostaje do niego przypięta.

Zmienna w Pythonie to **etykieta** przypięta do obiektu, a nie pudełko, do którego coś wkładamy.

```
price = 211.34
gross_price = price * 1.23
```

Metafora etykiety wróci w bloku 1.4 przy aliasowaniu list. Warto ją tu mocno ugruntować.

### Każdy obiekt ma typ i tożsamość

```
x = 10
type(x)        # <class 'int'>
id(x)          # liczba identyfikująca obiekt

x = "dziesięć"
type(x)        # <class 'str'>
```

Ta sama nazwa może wskazywać kolejno obiekty różnych typów: to **typowanie dynamiczne**.

### Nazwy zmiennych

- litery, cyfry i `_`; nie mogą zaczynać się od cyfry
- wielkość liter ma znaczenie: `Age` i `age` to różne nazwy
- nie mogą być słowami kluczowymi (`if`, `for`, `class`…)

```
import keyword
print(keyword.kwlist)
```

Polskie litery w nazwach są dozwolone (`cena_końcowa`), ale w praktyce unikamy ich w kodzie.

### Liczby całkowite: `int`

```
2 ** 100        # 1267650600228229401496703205376
10 / 3          # 3.3333333333333335  (zawsze float)
10 // 3         # 3   dzielenie całkowite
10 % 3          # 1   reszta z dzielenia
-7 // 2         # -4  zaokrąglenie w dół, nie w stronę zera
1_000_000       # podkreślenia dla czytelności
```

Python nie ma limitu wielkości liczb całkowitych: liczba może mieć tyle cyfr, ile zmieści się w pamięci.

### Liczby zmiennoprzecinkowe: `float`

```
0.1 + 0.2              # 0.30000000000000004
0.1 + 0.2 == 0.3       # False
```

Dlaczego? `float` to liczba binarna (IEEE 754, ok. 15–16 cyfr znaczących). Ułamka 0,1 nie da się zapisać w systemie dwójkowym dokładnie, tak jak 1/3 w dziesiętnym.

```
import math
math.isclose(0.1 + 0.2, 0.3)   # True
```

Kwoty pieniężne: typ `Decimal` (sesja 3), nie `float`.

### Więcej o `float`

```
1.5e-3             # 0.0015  zapis naukowy
float("inf")       # nieskończoność
round(2.675, 2)    # 2.67  (a nie 2.68!)
round(2.5)         # 2     zaokrąglenie bankierskie
round(3.5)         # 4
```

Zaokrąglenie bankierskie: wartości „po środku” zaokrąglane do liczby parzystej.

round(2.675, 2): 2.675 nie jest dokładnie reprezentowalne, w pamięci to 2.67499999… Do tego wrócimy przy Decimal.

### Wartości logiczne: `bool`

```
survived = True
age = 29
age >= 18          # True
age == 30          # False
age != 30          # True
```

Ciekawostka: `bool` to podtyp `int`.

```
True + True        # 2
sum([True, False, True, True])   # 3, czyli liczba ocalałych
```

### Tekst: `str`

```
first_name = "Elisabeth"
last_name = 'Allen'
description = """Tekst
w wielu liniach"""

first_name + " " + last_name  # 'Elisabeth Allen'
"=" * 20                      # '===================='
len(first_name)               # 9
first_name[0]                 # 'E'
```

Cudzysłów pojedynczy i podwójny są równoważne.

### Brak wartości: `None`

```
age = None          # wiek nieznany

age is None         # True
```

- `None` to osobny typ oznaczający **brak wartości**.
- W danych z Titanica brakuje wieku dla co piątego pasażera.
- Sprawdzamy przez `is None`, nie `== None`.

### Typowanie silne

Python nie zamienia typów „po cichu”:

```
"3" + 4             # TypeError
int("3") + 4        # 7
"3" + str(4)        # '34'
```

Konwersje mogą się nie udać:

```
int("3.5")          # ValueError
float("3,5")        # ValueError, przecinek dziesiętny!
float("3,5".replace(",", "."))   # 3.5
```

Przecinek dziesiętny: realny problem z plikami z polskiego Excela. Wróci w sesji 2 przy CSV.

### Operatory

| Rodzaj | Operatory |
|---|---|
| arytmetyczne | `+ - * / // % **` |
| porównania | `== != < <= > >=` |
| logiczne | `and or not` |
| przypisania | `= += -= *= /=` |

```
0 < age < 18        # porównania łańcuchowe
-2 ** 2             # -4: potęgowanie ma wyższy priorytet niż minus
counter += 1        # to samo co: counter = counter + 1
```

### Wejście i wyjście

```
first_name = input("Podaj imię: ")
age = int(input("Podaj wiek: "))    # input() zawsze zwraca str!

print("Pasażer:", first_name, age)
print("a", "b", "c", sep=";")       # a;b;c
print("bez nowej linii", end="")
```

### f-stringi

```
full_name = "Elisabeth Allen"
fare = 211.3375

f"{full_name} zapłaciła {fare} £"   # pełna wartość
f"{fare:.2f}"                       # '211.34'
f"{1234567.891:,.2f}"               # '1,234,567.89'
f"{full_name=}"                     # full_name='Elisabeth Allen'
```

Specyfikator po dwukropku: `.2f` to dwa miejsca po przecinku, `,` to separator tysięcy.

Separator tysięcy w f-stringu jest anglosaski (przecinek). Polski zapis: f”{x:,.2f}“.replace(”,“,” “).replace(”.”, “,”), albo moduł locale. Nie komplikować, tylko zasygnalizować.

### Zmienne i niezmienne

| Niezmienne (*immutable*) | Zmienne (*mutable*) |
|---|---|
| `int`, `float`, `bool`, `str`, `tuple` | `list`, `dict`, `set` |

```
s = "Allen"
s[0] = "a"          # TypeError: str nie obsługuje zmiany elementu
s = "allen"         # to nie zmiana, tylko nowy obiekt
```

Różnica stanie się ważna za chwilę, przy listach.

### Gdy coś pójdzie nie tak

Python zgłasza błędy na dwa sposoby:

| | Błąd składni | Wyjątek |
|---|---|---|
| Kiedy | przed uruchomieniem | w trakcie działania |
| Przyczyna | kod nie jest poprawnym Pythonem | poprawny kod natrafia na problem |
| Skutek | program w ogóle nie startuje | program zatrzymuje się w miejscu błędu |

Komunikat o błędzie to wskazówka, nie porażka. Zawsze czytamy go do końca.

### Błąd składni

```
age = 20
if age > 18
    print("dorosły")
```

Wynik uruchomienia:

```
  File ".../python-course/syntax.py", line 2
    if wiek > 18
               ^
SyntaxError: expected ':'
```

- Plik i numer linii wskazują miejsce.
- Znak ^ pokazuje, gdzie interpreter się zgubił.
- Ostatnia linia mówi, czego brakuje.

### Jak czytać komunikat o wyjątku

```
age = int("dwadzieścia")
```

Wynik uruchomienia:

```
Traceback (most recent call last):
  File ".../python-course/passenger.py", line 1, in <module>
    wiek = int("dwadzieścia")
           ~~~^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'dwadzieścia'
```

Czytamy **od dołu**: typ i opis (*co*), plik i linia (*gdzie*), podkreślony fragment (*który dokładnie*).

### Najczęstsze wyjątki

| Wyjątek | Typowa przyczyna | Przykład |
|---|---|---|
| `NameError` | literówka w nazwie, zmienna nieutworzona | `print(pirce)` |
| `TypeError` | operacja na niepasującym typie | `"3" + 4` |
| `ValueError` | dobry typ, zła wartość | `int("abc")` |
| `KeyError` | brak klucza w słowniku | `passenger["cabin"]` |
| `IndexError` | indeks poza listą | `[1, 2][5]` |
| `ZeroDivisionError` | dzielenie przez zero | `1 / 0` |
| `IndentationError` | niespójne wcięcia | |

### Przechwytywanie wyjątku: `try` / `except`

```python
text = input("Podaj wiek: ")
try:
    age = int(text)
except ValueError:
    print(f"„{text}" to nie jest liczba całkowita.")
    age = None

print("wiek:", age)
```

- Gdy w `try` wystąpi `ValueError`, wykonuje się blok `except`, a program działa dalej.
- Inne wyjątki nadal zatrzymują program.

### Dobre praktyki

- Przechwytujemy **konkretny** wyjątek, którego się spodziewamy.
- Nie używamy pustego `except:`: ukrywa też literówki i błędy logiki.
- Nie „wyciszamy” błędu bez reakcji (`except ValueError: pass`).
- Błąd danych wejściowych obsługujemy; błąd w kodzie poprawiamy.

```python
import this     # "Errors should never pass silently."
```

Rozwinięcie w sesji 2 (blok 2.1): else, finally, raise, walidacja danych z pliku titanic3. Pętla „pytaj aż do skutku” (while True + try/except + break) pasuje do bloku 1.6.

### Pokaz w VS Code

Program `passenger.py`:

- zapytać o imię, wiek i cenę biletu (`input`)
- przeliczyć cenę na liczbę (`float`, z obsługą przecinka)
- wypisać komunikat f-stringiem, cena z dwoma miejscami po przecinku
- sprawdzić `type()` każdej zmiennej
- celowo wywołać `TypeError` i `ValueError`, przeczytać komunikat błędu
- obsłużyć błędny wiek przez `try` / `except ValueError`

## 1.4 Kolekcje

### Cztery podstawowe kolekcje

| Typ | Zapis | Uporządkowana | Zmienna | Duplikaty |
|---|---|---|---|---|
| `list` | `[1, 2, 2]` | tak | tak | tak |
| `tuple` | `(1, 2, 2)` | tak | nie | tak |
| `dict` | `{"a": 1}` | tak (kolejność wstawiania) | tak | klucze unikalne |
| `set` | `{1, 2}` | nie | tak | nie |

### Lista

```
ages = [29, 0.92, 2, 30, 25]

ages[0]            # 29      indeksy od zera
ages[-1]           # 25      od końca
len(ages)          # 5
48 in ages         # False

ages.append(48)    # dodaj na końcu
ages.remove(2)     # usuń pierwsze wystąpienie
ages.sort()        # sortuj w miejscu
sorted(ages)       # nowa, posortowana lista
```

### Wycinki

```
letters = ["a", "b", "c", "d", "e", "f"]
#          0    1    2    3    4    5
#         -6   -5   -4   -3   -2   -1

letters[1:4]       # ['b', 'c', 'd']   od 1 do 4, bez 4
letters[:3]        # ['a', 'b', 'c']
letters[3:]        # ['d', 'e', 'f']
letters[::2]       # ['a', 'c', 'e']   co drugi
letters[::-1]      # odwrócona
```

Zasada: `[start:stop:krok]`, przy czym `stop` nie należy do wyniku.

### Krotka

```
point = (52.4, 16.9)          # niezmienna para
lat, lon = point              # rozpakowanie

a, b = 1, 2
a, b = b, a                   # zamiana bez zmiennej pomocniczej
```

Krotka nadaje się do danych, które logicznie stanowią całość i nie powinny się zmieniać.

### Słownik: rekord danych

```
passenger = {
    "name": "Allen, Miss. Elisabeth Walton",
    "pclass": 1,
    "survived": True,
    "age": 29.0,
    "fare": 211.3375,
    "port": "S",
}

passenger["age"]                    # 29.0
passenger["cabin"]                  # KeyError
passenger.get("cabin", "brak")      # 'brak'
passenger["cabin"] = "B5"           # dodanie klucza
```

Pierwszy wiersz zbioru titanic3.

### Operacje na słowniku

```
passenger.keys()    # nazwy pól
passenger.values()  # wartości
passenger.items()   # pary (klucz, wartość)

"age" in passenger  # True, sprawdza klucze
len(passenger)      # liczba pól
del passenger["port"] # usunięcie pola
```

Lista słowników to już tabela danych: każdy słownik jest jednym wierszem.

### Zbiór

```
ports = {"S", "C", "Q", "S", "S"}
ports               # {'S', 'C', 'Q'}, duplikaty znikają

a = {1, 2, 3}
b = {2, 3, 4}
a | b               # {1, 2, 3, 4}  suma
a & b               # {2, 3}        część wspólna
a - b               # {1}           różnica
```

Zastosowanie: unikalne wartości, szybkie sprawdzanie przynależności.

### Pułapka: aliasowanie

```
1 a = [1, 2, 3]
2 b = a
3 b.append(4)
4 print(a)          # [1, 2, 3, 4]
```

`b = a` nie kopiuje listy: przypina **drugą etykietę** do tego samego obiektu.

```
a is b              # True:  ten sam obiekt
c = a.copy()        # prawdziwa kopia
c == a              # True:  równe wartości
c is a              # False: różne obiekty
```

Najważniejszy slajd bloku. Narysować na tablicy: jeden obiekt-lista, dwie etykiety a i b.

### Pokaz w VS Code

- Zapisać pasażera jako słownik i odczytać kilka pól.
- Dodać pole `cabin`, porównać `[]` i `.get()` dla brakującego klucza.
- Zbudować listę 3–4 pasażerów (lista słowników).
- Powtórzyć pułapkę aliasowania, potem naprawić ją przez `.copy()`.
- Zbiór portów zaokrętowania z listy pasażerów.

## 1.5 Instrukcje warunkowe

### `if`, `elif`, `else`

```
age = 29
if age is None:
    group = "nieznany"
elif age < 18:
    group = "dziecko"
elif age < 60:
    group = "dorosły"
else:
    group = "senior"
```

- Dwukropek kończy warunek.
- **Wcięcie** (4 spacje) wyznacza blok: to część składni, nie ozdoba.
- Sprawdzane są kolejne warunki; wykonuje się **pierwszy** spełniony.

### Łączenie warunków

```
if woman and pclass == 1:
    ...
if port == "C" or port == "Q":
    ...
if not survived:
    ...
```

**Wartościowanie skrócone:** `and` przestaje sprawdzać po pierwszym fałszu.

```
if age is not None and age < 18:     # bezpieczne
    ...
if age < 18 and age is not None:     # TypeError, gdy age is None
    ...
```

### Prawdziwość wartości

W warunku każdą wartość można potraktować jak `bool`.

Fałszywe są: `False`, `None`, `0`, `0.0`, `""`, `[]`, `{}`, `set()`.

Wszystko inne jest prawdziwe.

```
passengers = []
if passengers:
    print("Mamy dane")
else:
    print("Lista pusta")
```

### Wyrażenie warunkowe

```
status = "ocalał" if survived else "zginął"
```

Zwięzła forma dla prostego wyboru jednej z dwóch wartości.

### Przykład: próg podatkowy

```
income = 150_000

if income <= 120_000:
    tax = 0.12 * income - 3600
else:
    tax = 10_800 + 0.32 * (income - 120_000)

tax = max(tax, 0)
print(f"Podatek: {tax:,.2f} zł")
```

Uproszczona skala PIT (12% i 32%, kwota zmniejszająca podatek 3600 zł), bez ulg i składek.

### `match` (Python 3.10+)

```
match port:
    case "S":
        name = "Southampton"
    case "C":
        name = "Cherbourg"
    case "Q":
```

```
        name = "Queenstown"
    case _:
        name = "nieznany"
```

`case _` łapie wszystkie pozostałe przypadki.

### Pokaz w VS Code

- Grupa wiekowa pasażera, z obsługą `None`.
- Kategoria ceny biletu: do 10 £ tani, do 50 £ średni, powyżej drogi.
- Pełna nazwa portu przez `match`.
- Zamienić kolejność warunków w `and` i zobaczyć `TypeError`.

## 1.6 Pętle

### Pętla `for`

```
ages = [29, 0.92, 2, 30, 25]

for age in ages:
    print(age)
```

`for` przechodzi po kolejnych elementach dowolnej **kolekcji**: listy, napisu, słownika, pliku…

```
for letter in "Titanic":
    print(letter)
```

### `range`, `enumerate`, `zip`

```
for i in range(5):              # 0, 1, 2, 3, 4
    ...
for i in range(2, 10, 2):       # 2, 4, 6, 8
    ...

for position, age in enumerate(ages, start=1):
    print(position, age)
```

```
classes = [1, 1, 1, 1, 1]
for pclass, age in zip(classes, ages):
    print(pclass, age)
```

### Po słowniku

```
for key in passenger:
    print(key)

for key, value in passenger.items():
    print(f"{key:>10}: {value}")
```

### Pętla `while`

Za ile lat kapitał się podwoi przy 5% rocznie?

```
capital = 1000.0
years = 0

while capital < 2000:
    capital *= 1.05
    years += 1

print(years)       # 15
```

`while` powtarza blok, dopóki warunek jest prawdziwy. Uwaga na pętle nieskończone.

### `break`, `continue`, `else`

```
for p in passengers:
    if p["age"] is None:
        continue                # pomiń, przejdź do następnego
    if p["age"] < 1:
        print("Znaleziono niemowlę")
        break                   # przerwij pętlę
else:
    print("Brak niemowląt")     # tylko gdy nie było break
```

### Wzorzec: akumulator

```
total = 0
count = 0

for p in passengers:
    if p["age"] is not None:
        total += p["age"]
        count += 1

average = total / count if count else None
```

Zmienna, która „zbiera” wynik w kolejnych obrotach pętli.

### Wzorzec: maksimum

```
oldest = None

for p in passengers:
    age = p["age"]
    if age is None:
        continue
    if oldest is None or age > oldest:
        oldest = age
```

To samo z funkcją wbudowaną:

```
max(p["age"] for p in passengers if p["age"] is not None)
```

### Wzorce: zliczanie i filtrowanie

```
# zliczanie: ilu pasażerów w każdej klasie
in_class = {}
for p in passengers:
    pclass = p["pclass"]
    in_class[pclass] = in_class.get(pclass, 0) + 1
```

```python
# filtrowanie: tylko ocalali
survivors = []
for p in passengers:
    if p["survived"]:
        survivors.append(p)
```

### Wyrażenia listowe

Wzorzec „przekształć i odfiltruj” w jednej linii:

```python
ages = [p["age"] for p in passengers if p["age"] is not None]

survivors = [p for p in passengers if p["survived"]]

squares = [x ** 2 for x in range(10)]
```

Słownikowe:

```python
port_names = {"S": "Southampton", "C": "Cherbourg", "Q": "Queenstown"}
codes = {name: code for code, name in port_names.items()}
```

### Ile kosztuje `in`?

```python
import timeit

number_list = list(range(1_000_000))
number_set = set(number_list)

timeit.timeit(lambda: 999_999 in number_list, number=100)  # setki ms
timeit.timeit(lambda: 999_999 in number_set, number=100)   # mikrosekundy
```

- Lista: sprawdzanie element po elemencie, czas rośnie z rozmiarem.
- Zbiór: odnalezienie elementu od razu, przez funkcję skrótu (hash).

Wyniki zależą od maszyny; pokazać na żywo, liczy się rząd wielkości różnicy.

## Pokaz w VS Code

- Średni wiek pasażerów z pominięciem braków.
- Liczba pasażerów w każdej klasie (słownik zliczający).
- Lista ocalałych przez pętlę, a potem przez wyrażenie listowe.
- Pętla `while`: po ilu latach kapitał się podwoi przy 3%, 5% i 7%?
- Porównanie czasu `in` dla listy i zbioru.

## Podsumowanie

- Program to algorytm zapisany w języku programowania; Python kompiluje do bytecode'u i interpretuje.
- Zmienna to etykieta przypięta do obiektu; każdy obiekt ma typ.
- `float` jest przybliżeniem; pieniądze liczymy inaczej.
- Kolekcje: lista, krotka, słownik, zbiór; lista słowników to tabela.
- Warunki i pętle sterują przebiegiem programu.